



Introduction to Assembly Language

Ittipon (Art) Rassameeroj, Ph.D.



Agenda

- What is Assembly language?
- Disadvantages and advantages
- Assembly language elements
- Intel x86 program execution registers
- Statement types
- Assemblers
- Assembly Programming
 - Defining data
 - Data transfer and addressing
 - Addition and subtraction
 - Multiplication and division
 - Indirect and indexed operands
 - JMP, Loop, CMP
 - Conditional structures
 - Compound expression
 - While loop
 - Procedures and stack operations
- Assembly program examples
- Compiling and running Assembly



Assembly Language

- One step away from machine language.
- Each assembly language instruction is translated into one machine instruction by the assembler.
- It is hardware dependent.
- Programmers rarely use assembly language for applications or even systems programs.



Disadvantages of Assembly Language

- Development time
- Reliability and security
- Debugging and verifying
- Maintainability
- Portability
- System code can use intrinsic functions instead of assembly.
- Application code can use intrinsic functions or vector classes instead of assembly.
- Compiler have been improved a lot in recent years.



Advantages of Assembly Language

- Debugging and verifying
- Making compilers
- Embedded systems
- Hardware drivers and system code
- Accessing instructions that are not accessible from high-level language.
- Self-modification code
- Optimizing code for size and speed
- Function libraries
- Making function libraries compatible with multiple compilers and OSs.



Assembly Language Elements

Label	Mnemonic	Operand(s)	;comment
Optional	Opcode name or directive name or macro name	Zero or more	Optional

```
L2: SUB EAX, EDX    ;subtract contents of register EDX from
                   ;contents of EAX and store result in EAX
    JG  L2          ;jump to L2 if result of subtraction is
                   ;positive
```



Statement Types

- Instructions
- Directives
 - Called pseudo-instructions.
 - Assembly language statements that are not directly translated into machine language instructions.
 - They are instruction to the assembler to perform specified actions doing the assembly process.
- Macro definitions



Directives

Unit	Letter
byte	B
word (2 bytes)	W
double word (4 bytes)	D
quad word (8 bytes)	Q
ten bytes	T

Name	Description	Example
DB, DW, DD, DQ, DT	Initialize locations	L6 DD 1A92H ;doubleword at L6 initialized to 1A92H
RESB, RESW, RESD, RESQ, REST	Reserve uninitialized locations	BUFFER RESB 64 ;reserve 64 bytes starting at BUFFER
INCBIN	Include binary file in output	INCBIN "file.dat" ; include this file
EQU	Define a symbol to a given constant value	MSGLEN EQU 25 ;the constant MSGLEN equals decimal 25
TIMES	Repeat instruction multiple times	ZEROBUF TIMES 64 DB 0 ;initialize 64-byte buffer to all zeros



Macro Definitions

```
%DEFINE B(X) = 2 * X
```

```
%DEFINE A(X) = 1 + B(X)
```

```
...
```

```
MOV AX, A(8)
```

```
%MACRO PROLOGUE 1
```

```
    PUSH EBP
```

```
    MOV EBP, ESP
```

```
    SUB ESP, %1
```

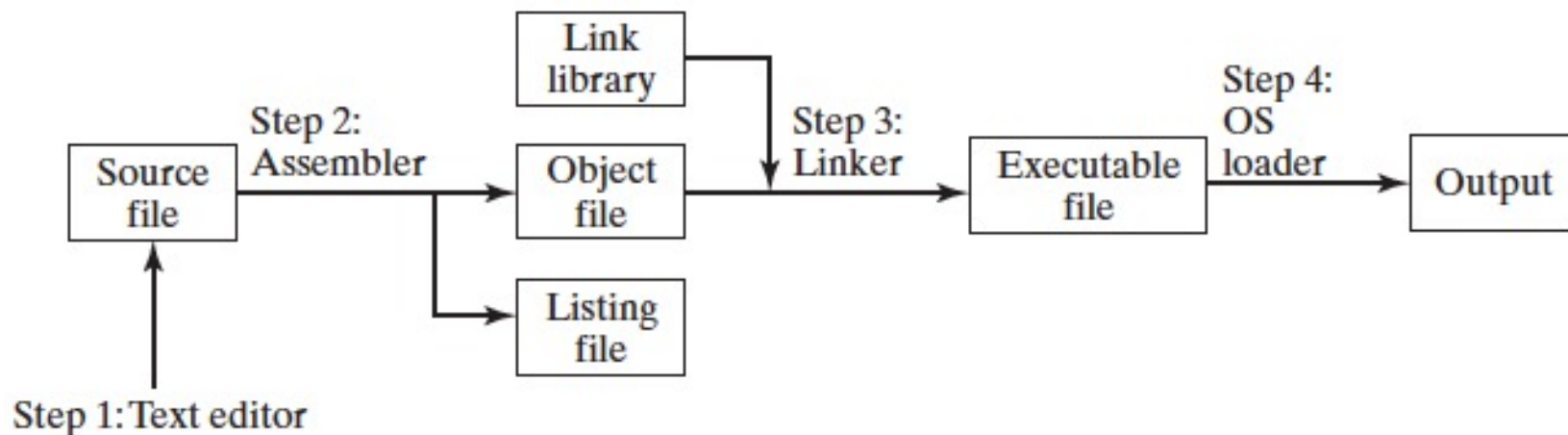
```
...
```

```
MYFUNC: PROLOGUE 12
```



Assemblers

- It is a software utility that takes an assembly program as input and produces object code (binary file) as output.
- There are two general approaches to assemblers: the two-pass assembler and one-pass assembler.
- Two-pass assembler is more common and easier to understand.





An Example of Listing File

```
1:      ; AddTwo.asm - adds two 32-bit integers.
2:      ; Chapter 3 example
3:
4:      .386
5:      .model flat,stdcall
6:      .stack 4096
7:      ExitProcess PROTO,dwExitCode:DWORD
8:
9:      00000000          .code
10:     00000000          main PROC
11:     00000000 B8 00000005      mov  eax,5
12:     00000005 83 C0 06        add  eax,6
13:
14:                                invoke ExitProcess,0
15:     00000008 6A 00          push  +000000000h
16:     0000000A E8 00000000 E    call  ExitProcess
17:     0000000F          main ENDP
18:                                END main
```



Assemblers

- First pass
 - Concerned with only label definitions.
 - Constructs a symbol table that contains a list of all labels and their associated location counter (LC) values.
 - It places the label into the symbol table, along with the current LC value.
 - The assembler continues until it has read all of the assembly language statements.
 - Examines each assembly statement and determines the length of the corresponding machine instruction and therefore how much to increment the LC.



Assemblers

- Second pass: translate each instruction into binary machine code.
 - Translate the mnemonic into binary opcode.
 - Use the opcode to determine the format of the instruction and the location and length of the various fields in the instruction.
 - Translate each operand name into the appropriate register or memory code.
 - Translate each immediate value into a binary string.
 - Translate any references to labels into the appropriate LC value using the symbol table.
 - Set any other bits in the instruction that are needed, including addressing mode indicators, condition code bits, and so on.

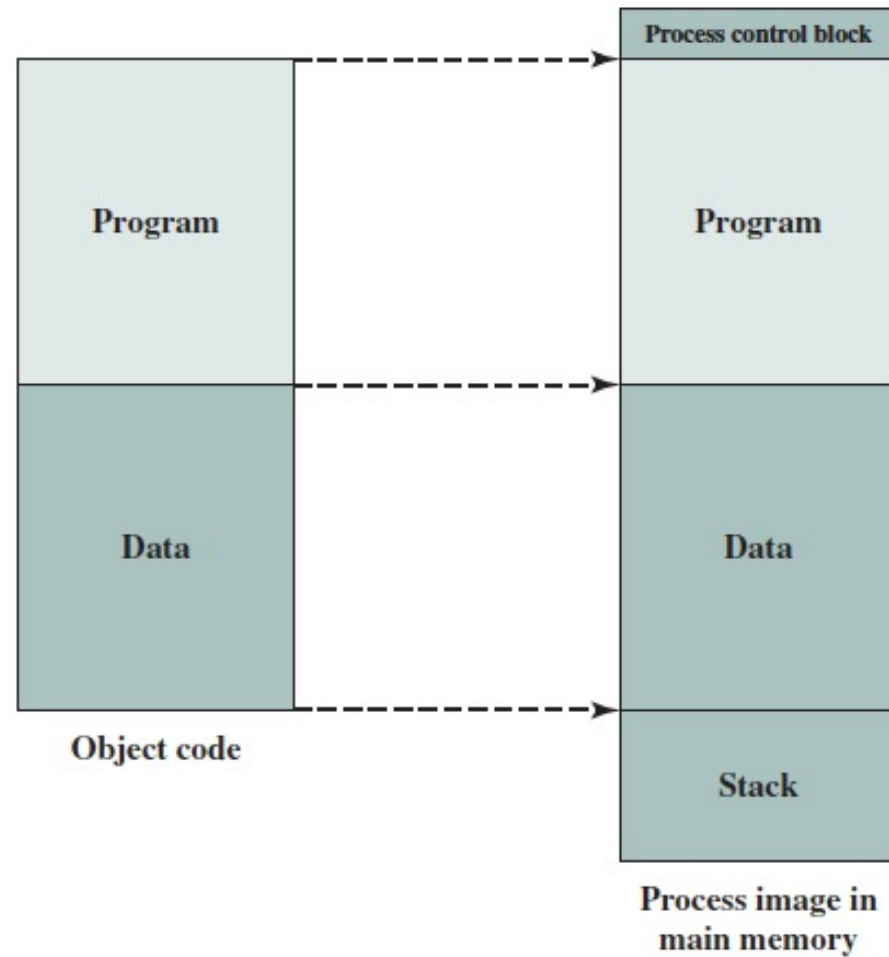


Assembler

- Zeroth pass
 - Read all macro definitions.
 - Typically, the assembly language requires that all macro definitions must appear at the beginning of the program.
 - Once all the macro are recognized, the assembler goes through the source code and expands the macros with their associated parameters.

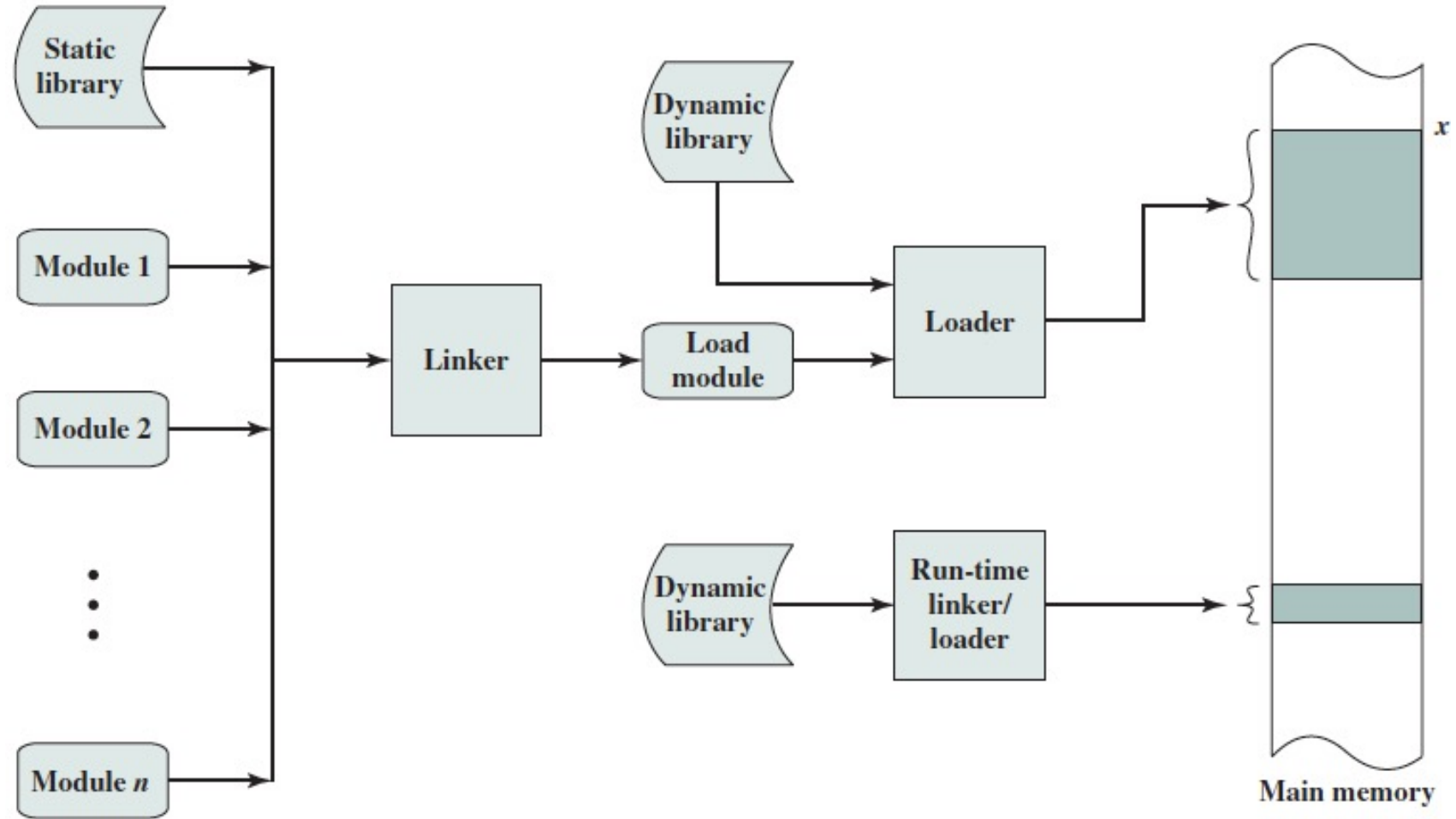


Loading



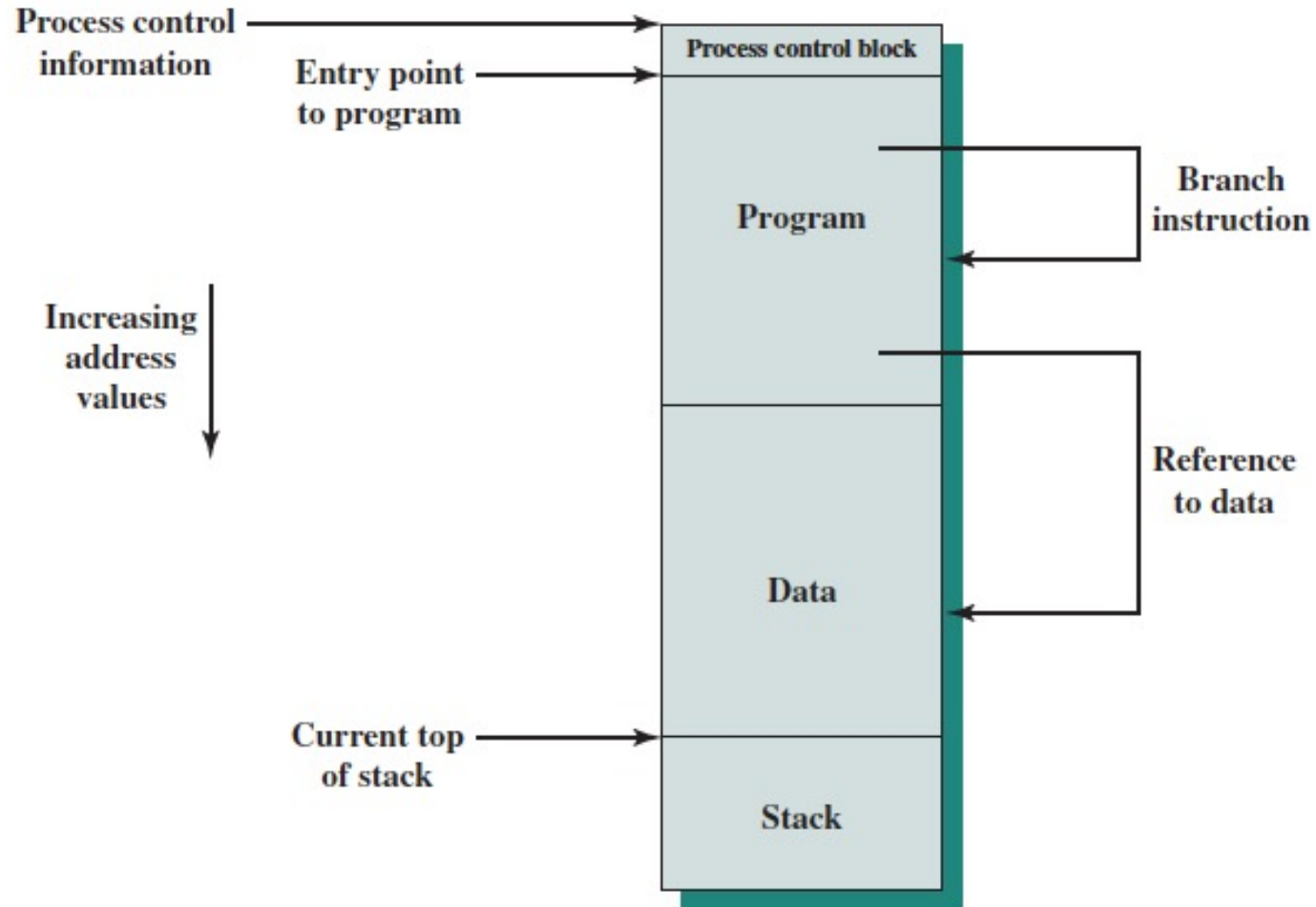


Linking and Loading



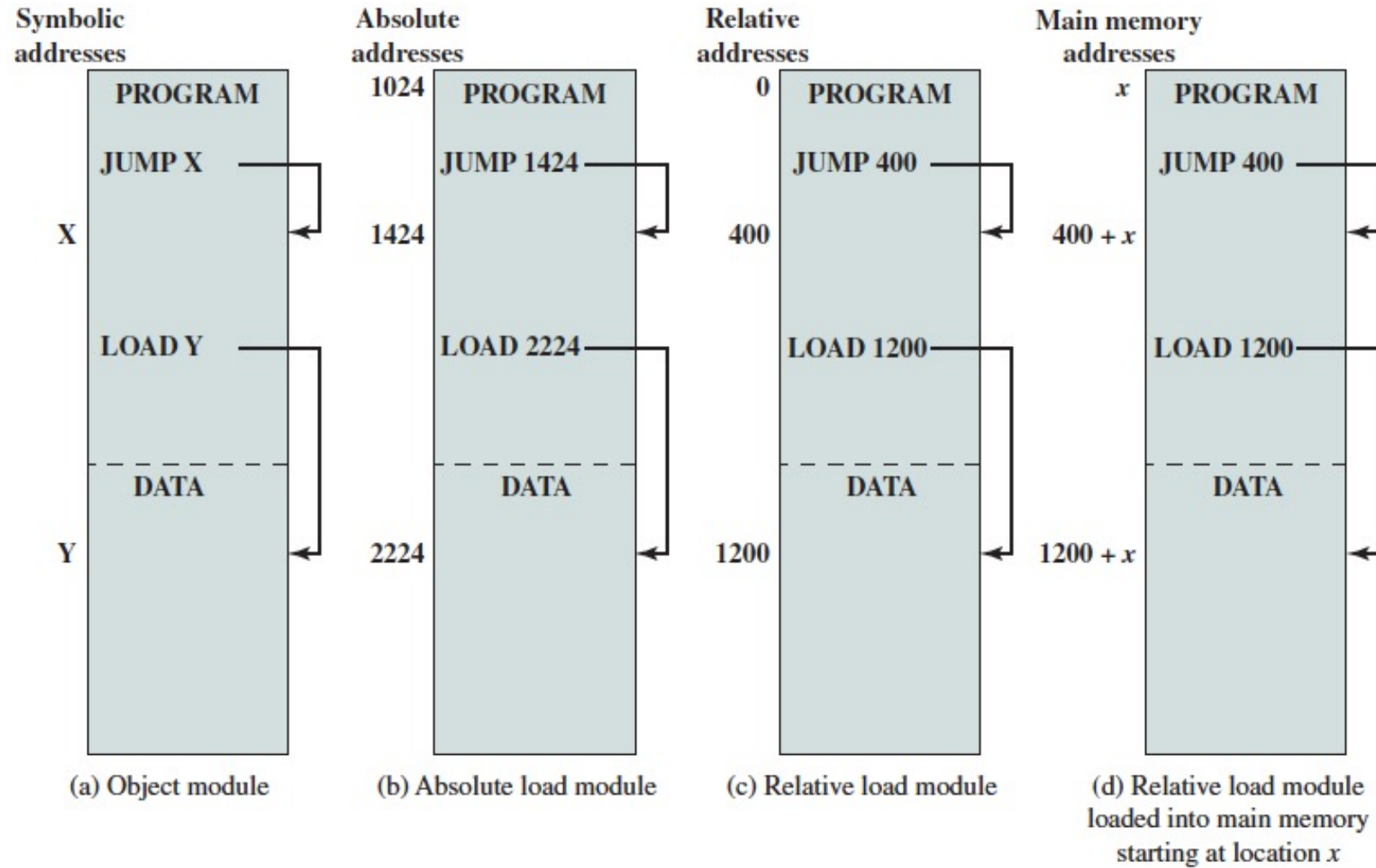


Addressing for a Process



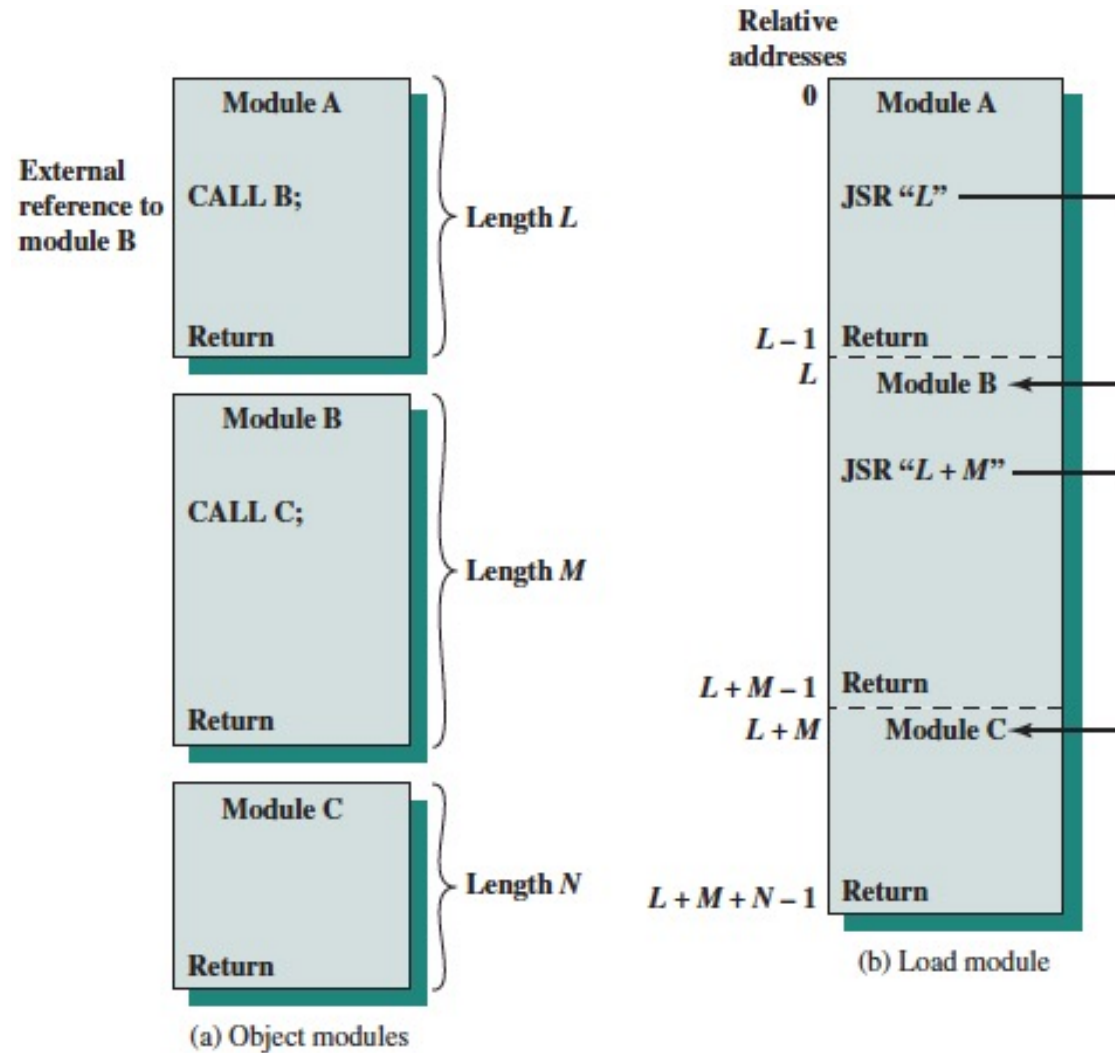


Loading





Linking Functions





Defining Data

BYTE	8-bit unsigned integer. B stands for byte
SBYTE	8-bit signed integer. S stands for signed
WORD	16-bit unsigned integer
SWORD	16-bit signed integer
DWORD	32-bit unsigned integer. D stands for double
SDWORD	32-bit signed integer. SD stands for signed double
FWORD	48-bit integer (Far pointer in protected mode)
QWORD	64-bit integer. Q stands for quad
TBYTE	80-bit (10-byte) integer. T stands for Ten-byte
REAL4	32-bit (4-byte) IEEE short real
REAL8	64-bit (8-byte) IEEE long real
REAL10	80-bit (10-byte) IEEE extended real

```

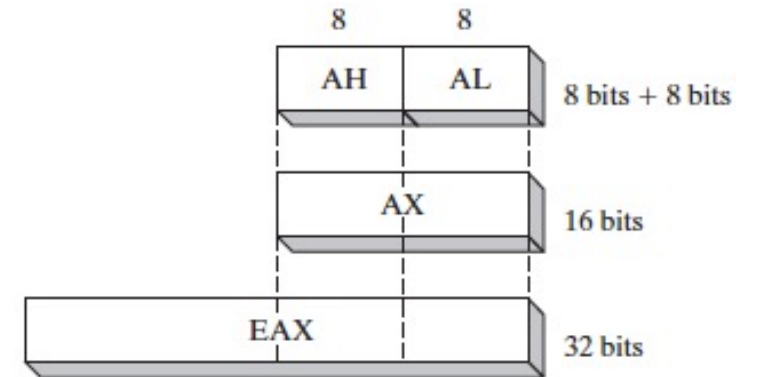
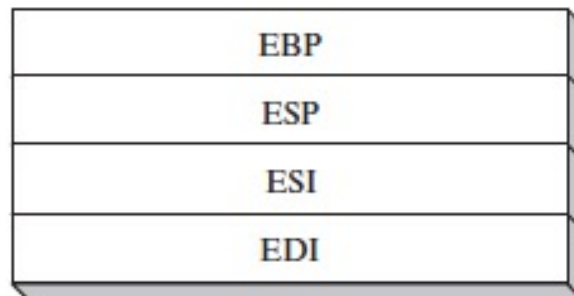
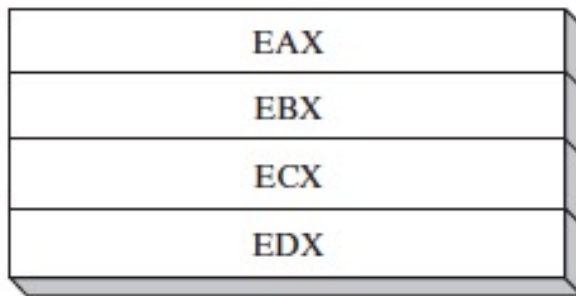
value1 BYTE 'A'           ; character literal
value2 BYTE 0              ; smallest unsigned byte
value3 BYTE 255            ; largest unsigned byte
value4 SBYTE -128          ; smallest signed byte
value5 SBYTE +127          ; largest signed byte
value6 BYTE ?
val1 DB 255
list1 BYTE 10,20,30,40
list2 BYTE 10,20,30,40
        BYTE 50,60,70,80
list3 BYTE 10, 32, 41h, 00100010b
list4 BYTE 0Ah, 20h, 'A', 22h
greeting BYTE "Good afternoon",0
value7 BYTE 20 DUP(0)
value8 BYTE 10 DUP(?)

```

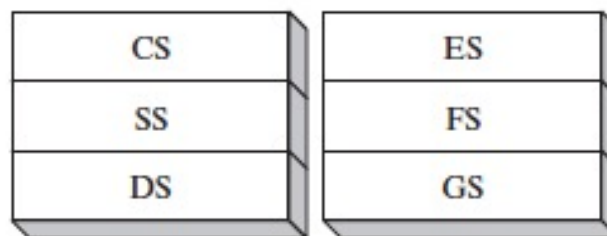
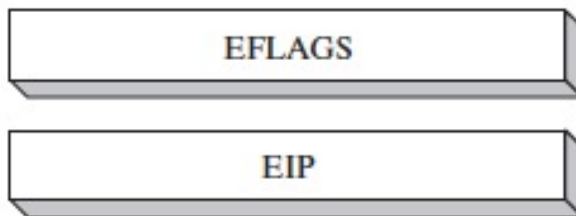


Intel x86 Program Execution Registers

32-Bit General-Purpose Registers



16-Bit Segment Registers



32-Bit	16-Bit
ESI	SI
EDI	DI
EBP	BP
ESP	SP



Defining Data: Floating-Point

rVal1 REAL4 -1.2

rVal2 REAL8 3.2E-260

rVal3 REAL10 4.6E+4096

ShortArray REAL4 20 DUP(0.0)

Data Type	Significant Digits	Approximate Range
Short real	6	1.18×10^{-38} to 3.40×10^{38}
Long real	15	2.23×10^{-308} to 1.79×10^{308}
Extended-precision real	19	3.37×10^{-4932} to 1.18×10^{4932}



Data Transfer: MOV

- Operand types: immediate (*imm*), register (*reg*), memory (*mem*)
- **MOV *destination, source***
- MOV instruction formats:
 - MOV *reg, reg*
 - MOV *mem, reg*
 - MOV *reg, mem*
 - MOV *mem, imm*
 - MOV *reg, imm*
- Direct memory operand:
var1 BYTE 10h
mov al var1
- Memory to memory transfer
mov ax, var1
mov var2, ax



Data Transfer: Direct Offset Operands

```
mov ecx, ebx
mov eax, 100h
mov eax, 1000000000b
mov eax, 256
mov ax, 1234h
mov [3158h], ax
```

Direct offset operands

```
.data
arrayA BYTE 10h, 20h, 30h, 40h
arrayB WORD 100h, 200h, 300h
.code
mov al, arrayA
mov al, [arrayA+1]
mov al, [arrayA+2]
mov ax, arrayB
mov ax, [arrayB+2]
```




Data Transfer: Indirect Operands

```
byteVal1 BYTE 10h  
mov esi, OFFSET byteVal1  
mov al, [esi]
```

```
arrayB BYTE 10h, 20h, 30h  
mov esi, OFFSET arrayB  
mov al, [esi]  
inc esi  
mov al, [esi]  
inc esi  
mov al, [esi]
```



Data Transfer: Indexed Operands

```
arrayB BYTE 10, 20, 30
```

```
mov esi, 0
```

```
mov al, arrayB[esi]
```

```
arrayW WORD 1000h, 2000h, 3000h
```

```
mov esi, OFFSET arrayW
```

```
mov ax, [esi]
```

```
mov bx, [esi+2]
```

```
mov cx, [esi+4]
```



Data Transfer: XCHG

- **XCHG**: exchanges the contents of two operands
 - `XCHG reg, reg`
 - `XCHG reg, mem`
 - `XCHG mem, reg`
- Examples:
 - `xchg ax, bx`
 - `xchg ah, al`
 - `xchg var1, bx`
 - `xchg eax, ebx`
- To exchange two memory operands
 - `mov ax, val1`
 - `xchg ax, val2`
 - `mov val1, ax`



Addition and Subtraction

- *INC reg/mem*
- *DEC reg/mem*
- *ADD dest, source*
- *SUB dest, source*
- *NEG reg/mem*

```
.data
myWord WORD 1000h
var1 DWORD 10000h
var2 DWORD 30000h
var3 DWORD 20000h
var4 SDWORD 26
.code
inc myWord
mov bx, myWord
dec bx
mov eax, var1
add eax, var2
sub eax, var3
mov ecx, var4
neg ecx
```

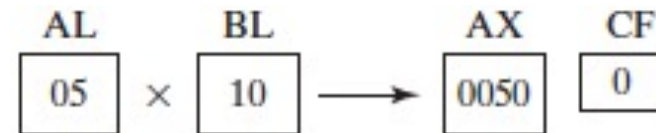


Multiplication

MUL *reg/mem* (multiplier)

Multiplicand	Multiplier	Product
AL	reg/mem8	AX
AX	reg/mem16	DX:AX
EAX	reg/mem32	EDX:EAX

```
mov al, 5h  
mov bl, 10h  
mul bl
```





Multiplication

.data

val1 WORD 2000h

val2 WORD 0100h

.code

mov ax, val1

mul val2



mov eax, 12345h

mov ebx, 1000h

mul ebx



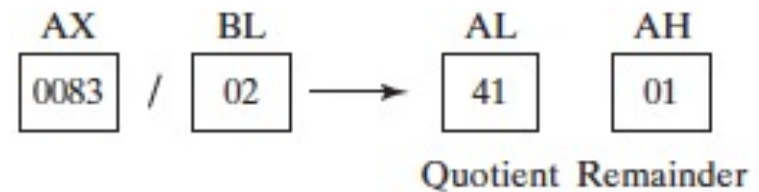


Division

DIV reg/mem (divisor)

Dividend	Divisor	Quotient	Remainder
AX	reg/mem8	AL	AH
DX:AX	reg/mem16	AX	DX
EDX:EAX	reg/mem32	EAX	EDX

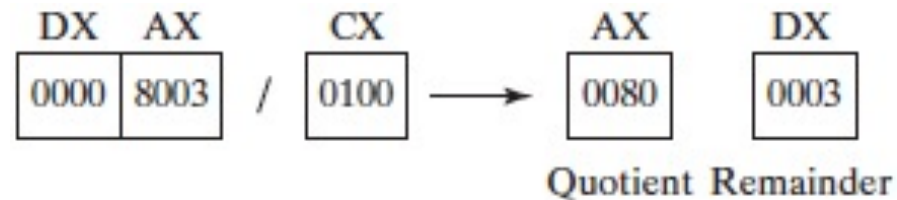
```
mov ax,0083h    ; dividend
mov bl,2         ; divisor
div bl
```





Division

```
mov dx,0           ; clear dividend, high
mov ax,8003h        ; dividend, low
mov cx,100h         ; divisor
div cx
```





JMP and LOOP

- *JMP dest_Label*

- Creating a loop:

`top:`

`...`

`...`

`jmp top`

- *LOOP dest_Label*

- $ECX = ECX - 1$

- If $ECX \neq 0$ then jump to *destination*

- Example:

```
mov ax, 0
```

```
mov ecx, 5
```

```
L1:
```

```
    inc ax
```

```
    loop L1
```



Nested Loop

```
.data
count DWORD ?
.code
    mov ecx, 100                ; set outer loop count
L1:
    mov count, ecx              ; save outer loop count
    mov ecx, 20                 ; set inner loop count
L2:
    ...
    ...
    loop L2                     ; repeat the inner loop
    mov ecx, count              ; restore outer loop count
    loop L1                     ; repeat the outer loop
```



CMP Instruction

- *CMP destination, source*
- The following pseudocode snippets support this instruction:
 - *if A > B ...*
 - *while X > 0 and X < 200 ...*
 - *if check_for_error(N) = true*

CMP Results	ZF	CF
Destination < source	0	1
Destination > source	0	0
Destination = source	1	0

CMP Results	Flags
Destination < source	SF ≠ OF
Destination > source	SF = OF
Destination = source	ZF = 1



Conditional Jumps with Status Flags

Mnemonic	Description	Flags / Registers
JZ	Jump if zero	ZF = 1
JNZ	Jump if not zero	ZF = 0
JC	Jump if carry	CF = 1
JNC	Jump if not carry	CF = 0
JO	Jump if overflow	OF = 1
JNO	Jump if not overflow	OF = 0
JS	Jump if signed	SF = 1
JNS	Jump if not signed	SF = 0
JP	Jump if parity (even)	PF = 1
JNP	Jump if not parity (odd)	PF = 0



Conditional Jumps

Mnemonic	Description
JE	Jump if equal ($leftOp = rightOp$)
JNE	Jump if not equal ($leftOp \neq rightOp$)
JCXZ	Jump if CX = 0
JECXZ	Jump if ECX = 0
JRCXZ	Jump if RCX = 0 (64-bit mode)

Mnemonic	Description
JA	Jump if above (if $leftOp > rightOp$)
JNBE	Jump if not below or equal (same as JA)
JAЕ	Jump if above or equal (if $leftOp \geq rightOp$)
JNB	Jump if not below (same as JAЕ)
JB	Jump if below (if $leftOp < rightOp$)
JNAЕ	Jump if not above or equal (same as JB)
JBE	Jump if below or equal (if $leftOp \leq rightOp$)
JNA	Jump if not above (same as JBE)

Mnemonic	Description
JG	Jump if greater (if $leftOp > rightOp$)
JNLE	Jump if not less than or equal (same as JG)
JGE	Jump if greater than or equal (if $leftOp \geq rightOp$)
JNL	Jump if not less (same as JGE)
JL	Jump if less (if $leftOp < rightOp$)
JNGE	Jump if not greater than or equal (same as JL)
JLE	Jump if less than or equal (if $leftOp \leq rightOp$)
JNG	Jump if not greater (same as JLE)



Conditional Jumps

- Example 1
`cmp eax, 0`
`jz L1`
.
.
`L1:`
- Example 2
`cmp eax, 5`
`je L1`

- Example 3
`mov ax, 5`
`cmp ax, 6`
`j1 L1`
- Example 4
`mov bx, +32`
`cmp bx, -35`
`jng L1`
`jnge L2`
`jge L3`



Conditional Structures

```
if(op1 == op2)
{
    x = 1;
    y = 2;
}
...
```

```
mov eax, op1
cmp eax, op2    ; op1 == op2?
jne L1          ; no: skip next
mov x, 1        ; yes: assign X and Y
mov y, 2
L1: ...
```

```
mov eax, op1
cmp eax, op2    ; op1 == op2?
je L1           ; yes: jump to L1
jmp L2          ; no: skip assignments
L1: mov x, 1    ; assign X and Y
     mov y, 2
L2: ...
```



Conditional Structures

```
if op1 > op2
    call Routine1
else
    call Routine2
end if
```

```
mov eax,op1
cmp eax,op2    ; op1 > op2?
jg A1          ; yes
call Routine2  ; no
jmp A2         ; exit
A1: call Routine1
A2: ...
```




Compound Expression

```
if (a1 > b1) AND (b1 > c1)
    x = 1
end if
```

```
cmp al,b1
ja L1
jmp next
L1:  cmp b1,c1
     ja L2
     jmp next
L2:  mov x,1
next:
```



While Loop

```
while(val1 < val2)
{
    val1++;
    val2--;
}
```

```
mov eax,val1
beginwhile:
    cmp eax,val2    ; if not (val1 < val2)
    jnl endwhile   ; exit the loop
    inc eax
    dec val2
    jmp beginwhile  ; repeat the loop
endwhile:
    mov val1,eax
```



Procedures

```
main PROC
```

```
...
```

```
...
```

```
main ENDP
```

```
sample PROC
```

```
...
```

```
...
```

```
ret
```

```
sample PROC
```

```
.data
```

```
theSum DWORD ?
```

```
.code
```

```
main PROC
```

```
    mov eax, 10000h
```

```
    mov ebx, 20000h
```

```
    mov ecx, 30000h
```

```
    call SumOf
```

```
    mov theSum, eax
```

```
main ENDP
```

```
SumOf PROC
```

```
    add eax, ebx
```

```
    add eax, ecx
```

```
    ret
```

```
SumOf PROC
```



Stack Operation

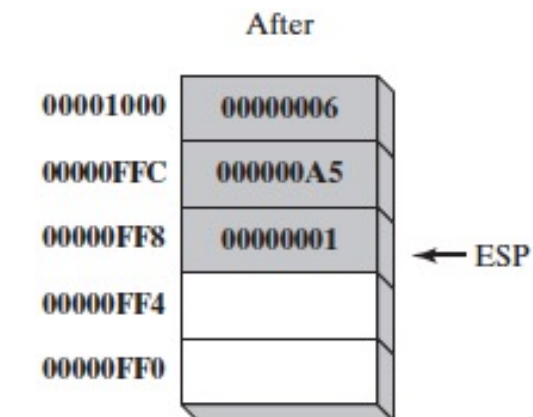
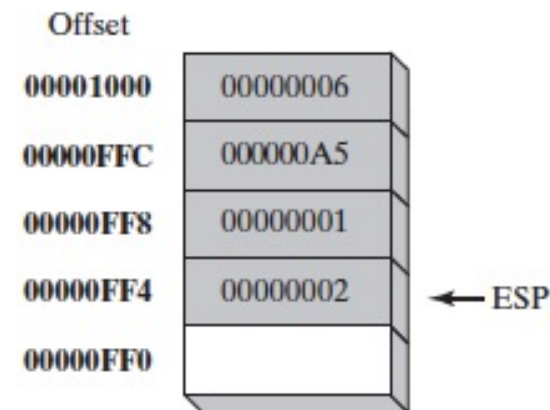
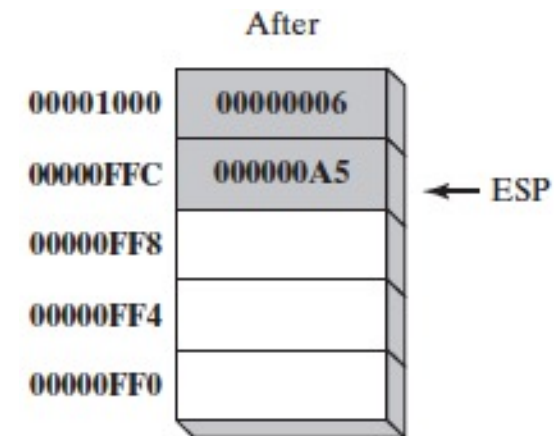
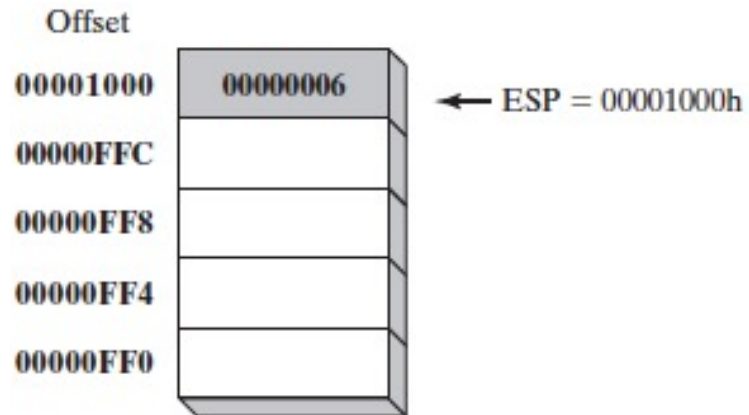
PUSH 00000006

PUSH 000000A5

PUSH 00000001

PUSH 00000002

POP





PUSH and POP Instructions

- Push and pop elements on the stack
 - **PUSH** *reg/mem/imm*
 - **POP** *reg/mem*
- **PUSHFD** and **POPFD** push and pop EFLAGS registers on the stack
- **PUSHAD** pushes and pop all of the 32-bit general-purpose registers on the stack in order: EAX, ECX, EDX, EBX, ESP (value before executing PUSHAD), EBP, ESI, and EDI.
- **POPAD** pops the same registers off the stack in reverse order.
- **PUSHA** pushes 16-bit general-purpose registers (AX, CX, DX, BX, SP, BP, SI, DI) on the stack in the order listed.
- **POPA** pops the same registers in reverse.



Example 1: GCD Program (Euclid's Algorithm)

```
unsigned int gcd (unsigned int a, unsigned int b)
{
    if (a == 0 && b == 0)
        b = 1;
    else if (b == 0)
        b = a;
    else if (a != 0)
        while (a != b)
            if (a < b)
                b -= a;
            else
                a -= b;
    return b;
}
```



Example 1: GCD Program

```

unsigned int gcd
(unsigned int a, unsigned int b)
{
    if (a == 0 && b == 0)
        b = 1;
    else if (b == 0)
        b = a;
    else if (a != 0)
        while (a != b)
            if (a < b)
                b -= a;
            else
                a -= b;
    return b;
}

```

gcd:	mov	ebx, eax	gcd:	neg	eax
	mov	eax, edx		je	L3
	test	ebx, ebx	L1:	neg	eax
	jne	L1		xchg	eax, edx
	test	edx, edx	L2:	sub	eax, edx
	jne	L1		jg	L2
	mov	eax, 1		jne	L1
	ret		L3:	add	eax, edx
L1:	test	eax, eax		jne	L4
	jne	L2		inc	eax
	mov	eax, ebx	L4:	ret	
	ret				
L2:	test	ebx, ebx			
	je	L5			
L3:	cmp	ebx, eax			
	je	L5			
	jae	L4			
	sub	eax, ebx			
	jmp	L3			
L4:	sub	ebx, eax			
	jmp	L3			
L5:	ret				



Example 2: Prime Number C Program

```
unsigned guess;
unsigned factor;
unsigned limit;

printf("Find primes up to : ");
scanf("%u", &limit);
printf("2\n");
printf("3\n");
guess = 5;
while(guess <= limit) {
    factor = 3;
    while(factor * factor < guess && guess % factor != 0)
        factor += 2;
    if (guess % factor != 0)
        printf("%d\n", guess);
    guess += 2;
}
```

/* current guess for prime */
/* possible factor of guess */
/* find primes up to this value */

/* treat first two primes as */
/* special case */
/* initial guess */
/* look for a factor of guess */

/* only look at odd numbers */



Example 2: Prime Number Assembly Program (1/2)

```
%include "asm_io.inc"

segment .data
Message db "Find primes up to: ", 0

segment .bss
Limit resd 1      ; find primes up to this limit
Guess resd 1      ; the current guess for prime

segment .text
    global _asm_main
_asm_main:
    enter 0,0      ; setup routine
    pusha
    mov eax, Message
    call print_string
    call read_int ; scanf("%u", & limit);
    mov [Limit], eax
```

```
    mov eax, 2      ; printf("2\n");
    call print_int
    call print_nl
    mov eax, 3      ; printf("3\n");
    call print_int
    call print_nl
    mov dword [Guess], 5 ; Guess = 5;
while_limit:        ; while (Guess <= Limit)
    mov eax, [Guess]
    cmp eax, [Limit]
    jnbe end_while_limit ; use jnbe since numbers
                        ; are unsigned
    mov ebx, 3      ; ebx is factor = 3;
```



Example 2: Prime Number Assembly Program (2/2)

```
while_factor:
    mov eax,ebx
    mul eax          ; edx:eax = eax*eax
    jo end_while_factor ; if answer won't
                     ; fit in eax alone

    cmp eax, [Guess]
    ; if !(factor*factor < guess)
    jnb end_while_factor
    mov eax,[Guess]
    mov edx,0
    div ebx          ; edx = edx:eax%ebx
    cmp edx, 0
    je end_while_factor ;if !(guess% factor != 0)
    add ebx,2        ; factor += 2;
    jmp while_factor
```

```
end_while_factor:
    je end_if        ;if !(guess% factor != 0)
    mov eax,[Guess]  ;printf("%u\n")
    call print_int
    call print_nl
end_if:
    add dword [Guess], 2 ;guess += 2
    jmp while_limit
end_while_limit:
    popa
    mov eax, 0        ;return back to C
    leave
    ret
```



Compiling Assembly

- On Windows
 - Download NASM from [Mycourses](#)
 - `nasm -f obj -d obj_type sourcecode.asm`
 - `bcc32 sourcecode.obj driver.obj asm_io.obj`
- For macOS
 - `$ brew install nasm`
 - `$ nasm -f macho64 sourcecode.asm`
 - `$ ld -o execute_file sourcecode.o`



References

